FilsPay Submission

Team: Caramel Popcorn

Member	Owned
Ali Abdulla	Security and money review, bug fixes across the API, settlement and reporting, refunds end to end, load hardening and load testing, AI setup, findings report and submission documents
Ali Aldoseri	Docker build and deployment fixes, and the visual design of checkout, the merchant dashboard and the admin console

Live URLs

Checkout	http://168.144.214.150/pay/<payment-link-token> (create a link in the dashboard to get a token)
Merchant dashboard	http://168.144.214.150/dashboard/login
Admin console	http://168.144.214.150:8080/login
Public API	http://168.144.214.150/v1/… (health: http://168.144.214.150/healthz)

Repository

Public fork: https://github.com/aliabdulla5/fils-pay
Last commit before the deadline: **[fill in at the deadline; latest code commit at the time of writing: `4e85bda`]**

1. Summary

What we found. The case file's 25 items split into 74 distinct defects. A second review pass found 20 more (inventory items 75–94). In total that is 94 findings in `docs/BUG_INVENTORY.md`: 81 confirmed and fixed, 10 confirmed and left open (most by explicit instruction, the rest needing DBA or design approval), and 1 rejected as intentional. The new ones include an unauthenticated reporting endpoint and a cross-merchant data leak. On the money side, a hold-expiry script deleted every hold on a quiet night, a capture replay double-credited a merchant's discount, and an admin re-run could close an unfinished business day and strand that day's later captures.

What we fixed. Every confirmed defect that the repository rules allow us to fix, each with the smallest change that works. Fixes are in the source and migrations; no test was edited.

What we left alone. Frozen code (`packages/money`, the Go money mirror), locked tests, the v1 partner API's 501 stubs, the deferred refund webhook, the documented `Math.random()` and display-name rules, and anything needing a DBA-approved migration (widening the platform volume counter, replacing the REAL lifetime estimate).

Main root cause. The same fact is computed in two places that drifted apart, and nothing checks that they agree. Examples:

- The balance button versus the ledger walk.
- Capture-time fees versus settlement fees.
- The analytics rebuild versus the reporting worker.
- Webhook delivery being used to write a payout record.
- The "docs are truth" agent files contradicting the ADRs.

Locked tests that assert the buggy behaviour make the drift look correct.

2. Architecture and deployment

```
Internet ── http://168.144.214.150 (Dilmune Cloud server)
   │
   ▼
 edge (nginx) :80  web app: checkout + merchant dashboard, /v1 → api
              :8080 admin console, /v1 → api
              sheds load: max 48 api requests in flight, excess → JSON 503 + Retry-After
   │
   ▼
 api :3000 (127.0.0.1 only)  Fastify, 0.5 vCPU / 512 MB, pool 30, in-flight cap 100
   │  webhook dispatcher in-process (LISTEN/NOTIFY + poll)
   ▼
 postgres :5432 (127.0.0.1 only)
   ▲                        ▲
 settlement :8091           reporting (no port)
 internal only, bearer      outbox consumer
 daily close 00:15 Asia/Bahrain
```

Only the edge faces the internet. The api and Postgres ports are bound to the host's loopback, and settlement's internal API has no published port at all.

- **Hosting:** a Dilmune Cloud Ubuntu server. nginx serves the web app and `/v1` on :80 and the admin console with `/v1` on :8080, the same layout as the compose `edge` service (`ops/nginx.conf`). The demo is served over plain HTTP on an IP address; TLS needs a domain name. **[Confirm the live nginx includes the repo's load shedding (`ops/api-proxy.inc`: 48 in flight, JSON 503 + Retry-After), or run the compose `edge` service instead.]**
- **One-time codes:** merchant and admin login codes are emailed over SMTP (`apps/api/src/auth/email.ts`; `OTP_SMTP_HOST`, `OTP_SMTP_PORT`, `OTP_SMTP_SECURE`, `OTP_SMTP_USER`, `OTP_SMTP_PASSWORD`, `OTP_FROM_EMAIL` in `ops/.env`). In production, a missing or failing SMTP relay makes login answer 503 `otp_delivery_failed` and revokes the pending session; production never returns the code in the response. Only development returns `devOtpCode`, for local use. **[Live server, checked at commit 4e85bda: the login response still includes devOtpCode, so the api is not running with NODE_ENV=production. Switch it to production with the SMTP settings before judging.]**
- **Redeploy (one command):** `docker compose -f ops/docker-compose.yml up -d --build`, preceded on schema changes by `docker compose -f ops/docker-compose.yml run --rm migrate`. The `seed` job is behind a profile and never runs on `up`, because it truncates the ledger.
- **Secrets:** `ops/.env`, which is gitignored and read by compose for `${VAR}` substitution (template: `ops/.env.docker.example`). `NODE_ENV` defaults to `production`, and production refuses to boot without `SETTLEMENT_INTERNAL_TOKEN`, `ACQUIRER_CALLBACK_SIGNING_SECRET` and a 32-byte `WEBHOOK_SECRET_ENCRYPTION_KEY`. One acquirer signing secret was committed to git history in the past and must be rotated with the acquirer.
- **Health checks and restarts:** every service has `restart: unless-stopped`. The api has a container health check on `/healthz` (every 10 s), and the edge starts only once the api is healthy. Postgres has its own health check. Measured: after the api process was killed with `SIGKILL` mid-traffic, it was serving again **5 seconds** later (see section 3).
- **Redeploys and settlement runs:** each batch close and each payout is one Postgres transaction. On SIGTERM the daemon lets the in-flight transaction commit or roll back, and compose gives it `stop_grace_period: 60s` to do so. Close and pay are idempotent per (merchant, date), so the next tick or `settlement run-batch --date=…` finishes the day. Closing an unfinished business day is refused (finding F78).

3. Capacity

Estimate: about 50 checkouts per second (about 100 requests/s; one checkout is a page load plus a pay call) with p95 under 1 s, at around 40 concurrent buyers. Above that, the stack refuses the excess quickly instead of slowing everyone down.

Our load test. Tool: a Node load driver (`submission/evidence/load/checkout_load.mjs`). Each virtual buyer opens a fresh, single-use payment link (`GET /v1/payment-links/:token`) and pays it (`POST …/pay`), in a loop, with no think time. Target: the full compose stack through the edge, with the api limited to exactly **0.5 vCPU / 512 MB**, in production mode. The acquirer simulator adds 100–400 ms per call and declines or fails about 2% of cards on purpose (the 402/502 responses below).

Concurrent buyers	Paid / s	p50	p95	p99	Outcome
10	18.1	537 ms	797 ms	926 ms	all served
20	37.6	527 ms	770 ms	895 ms	all served
40	**55.5**	**704 ms**	**989 ms**	1.1 s	**capacity**: all served

60	46.2	763 ms	1.3 s	1.6 s	excess refused with 503
200 (5× overload)	24.0	1.0 s	2.5 s	3.0 s	excess refused with 503; **zero 500s**
back to 40	52.8	727 ms	1.2 s	1.4 s	recovered immediately

Where the bottleneck was, and what we changed:

1. **Database pool.** A checkout holds one connection across two acquirer calls, so the original pool of 5 capped throughput at about 10 checkouts/s; at 10 it measured 20/s. With a pool of 30 the api reaches its CPU limit (about 45–50% of one core in `docker stats`) at about 55/s.
2. **Ledger scans (finding F93).** The ledger's deferred balance trigger scanned the whole `ledger_entries` table four times per checkout, because `entry_group_id` had no index. That made checkouts slower as the ledger grew; one check took 39 ms at 68k rows. Migration 0080 adds the index: 0.26 ms.
3. **Overload.** Before, excess requests queued on the pool for 5 s and then failed with 500. Now the edge admits at most 48 api requests at once and answers the rest with an immediate JSON 503 and `Retry-After: 1`. The api has its own in-flight cap (100) as a backstop, and maps pool timeouts to 503. The edge also turns 502/504 during an api restart into the same retryable 503.

Restart and recovery. We killed the api's node process (`SIGKILL`) during a 20-buyer run. The restart policy brought it back and it was serving 5 seconds later. While it was down the edge answered 503. The checkouts in flight at the moment of the kill were left *reserved*, never captured, and became payable again after the 60 s reservation window; we paid one to confirm.

No data lost. After all runs (17,663 paid checkouts, overload stages and the kill) we checked the database:

- 0 double charges.
- 0 transactions attached to two links.
- 0 amount mismatches.
- 0 captured payments without a paid link.
- 0 unbalanced ledger groups.
- 0 paid links missing their ledger credit.

Caveats: the load generator ran on the same machine as the stack, so under 5× overload it competed with the stack for CPU (it generated thousands of rejected requests per second). Goodput at 200 buyers should be better with a remote generator. **[Re-run on the deployed server and replace these numbers.]**

4. Findings report

The full inventory, with all 94 entries and their status, is `docs/BUG_INVENTORY.md`. Items 1–74 come from the first rounds (the sibling review); they are summarised in the table at the end of this section. Below are the 20 findings from the second review pass, written out in full.

Method for every entry: we traced the path from input to side effect, checked the ADRs, handover and changelog for intended behaviour, reproduced the defect, applied the smallest fix, and verified it against a freshly migrated Postgres 16 database. We used an untouched copy of `HEAD` plus only our patch, so unrelated working-tree changes could not affect the results. The full suites were then re-run: no new failures (see Appendix).

F75 — Reporting endpoint with no authentication

Symptom: `GET /v1/reporting/export-summary` returned 500 for everyone. **Found/reproduced:** route map review; `app.inject` without credentials returned 500. **Root cause:** the route was registered without the `requireMerchantAuth` preHandler, so `request.merchantAuth!` was undefined. **Fix:** preHandler added. **Verified:** unauthenticated returns 401, authenticated returns 200. **Existing data:** none.

F76 — Cross-merchant transaction report (IDOR)

Symptom: any merchant could read a marketplace sub-merchant's transactions by putting `{"merchantId": "<child>"}` in the report filter. **Root cause:** `resolveReportMerchantId` checked only that the id was *some* parent's split child, not the caller's. **Fix:** require `parent_merchant_id = caller`. **Verified:** an unrelated merchant resolves to itself; the real parent still reaches its child. **Existing data:** none; audit access logs if exposure matters.

F77 — Payout disbursement record written by webhook delivery

Symptom: `settlement_payout_disbursements` got one row per active webhook endpoint and another per replay or archive restore, and none for merchants without an endpoint. **Root cause:** the record was a "pre-delivery effect" of the first `settlement.paid` delivery (attempt 0, which every replay starts at), contrary to migration 0057 ("written inside PayoutBatch's transaction, one row per paid batch"). **Fix:** written in `PayoutBatch` and `PayoutSplitBatch` inside the payout transaction; removed from the dispatcher. **Verified:** Go test against a real database: two payout calls produce exactly one row, carrying the batch's net. **Existing data:** historical duplicates and missing rows need a one-off reconciliation (proposed: keep the earliest row per batch, back-fill paid batches that have none). Not run.

F78 — Closing an unfinished business day strands captures

Symptom: an admin or CLI `run-batch` for today (or a future date) closed and paid the batch, and every capture later that day was never settled. **Root cause:** no guard; a closed batch makes the re-run a no-op, and the next day's window does not reach back. **Fix:** `ErrBusinessDayNotComplete` in `CloseBatch` and `RunDailyBatch`; the internal API returns 400 `business_day_not_complete`. The daemon is unaffected (it always closes yesterday). **Verified:** today and tomorrow are refused and leave no batch row; yesterday closes. **Existing data:** query for capture events with `settlement_batch_id IS NULL` older than one day to find any stranded funds.

F79 — Analytics rebuild drops refunds and double-counts

Symptom: after **Rebuild analytics**, refund totals were zero, and captures still waiting in the reporting worker's backlog were counted twice. **Root cause:** a second implementation that rebuilt from `transaction_events` instead of the outbox the worker consumes. **Fix:** rebuild from the outbox up to the worker's watermark, under the TRUNCATE lock, with the worker's own buckets. **Verified:** refunds kept; events past the watermark are not counted. **Existing data:** re-run the rebuild once.

F80 — Hold expiry deletes every hold

Symptom: on a night with no expired holds, `scripts/expire_holds.ts` ran `DELETE FROM holds` with no WHERE clause. **Root cause:** `buildWhere` silently dropped an empty array filter. **Fix:** an empty list now means `WHERE FALSE`; the script skips the delete when nothing matched, and runs in one transaction with `FOR UPDATE`. **Verified:** with one live hold and zero expired, nothing is deleted; with one expired, only that one. **Existing data:** holds already deleted cannot be restored from this table.

F81 — Refunded transactions crash the merchant dashboard

Symptom: the detail page of any refunded transaction, and the reconciliation page, failed to render. **Root cause:** the dashboard's `bhd()` called `toFils` without `allowNegative`, but refund timeline amounts are negative by design (migration 0044). **Fix:** signed formatting, as the admin console already does. **Verified:** `bhd(-4750)` returns `"-4.750 BHD"`.

F82 — Merchants logged out after 10 minutes

Root cause: OTP verify promoted the session to 12 h but never re-issued the cookie, which kept the pending session's 10-minute `maxAge`. **Fix:** re-issue the cookie, as admin verify does. **Verified:** the verify response sets `Max-Age=43200`.

F83, F84, F85, F87 — Bad input returned 500 instead of 4xx

Four separate inputs reached the database or Node unchecked: unknown profile-PATCH keys (`SET undefined = $2`); invalid transaction `from`/`to` (`"NaN-NaN…"` sent to Postgres, although the route documents 400); a KYC filename outside latin-1, e.g. Arabic (Node rejects the header value with `ERR_INVALID_CHAR`); and an unknown `businessType` (DB CHECK violation). **Fixes:** allow-list the patchable keys, validate dates, use an RFC 6266 `filename*` header, validate business types. **Verified:** each reproduced as a 500 first, then returns 400 or 200 after the fix.

F86 — Database published on all interfaces

Root cause: compose mapped `5432:5432` with the committed dev password, and the hosted box runs the same file. **Fix:** bound to `127.0.0.1`.

F88 — Capture replay double-credits a negotiated discount

Symptom/reproduction: merchant at 250 bps with a 10% discount: capture 100 fils, then 1,000,000 fils, then replay the first capture's idempotency key. The merchant was credited 2.500 BD for the second capture *twice*. **Root cause:** the guard compared "adjustments < capture events" and then adjusted the *latest* capture. A capture whose discount rounds to 0 never gets an adjustment, so the counts never catch up. **Fix:** match adjustments to the captures that are owed one, oldest first. **Verified:** the same reproduction now credits once. **Existing data:** look for transactions with more discount-adjustment groups than captures owed one.

F89 — Inbound acquirer signature checked over re-serialized JSON

Symptom: a correctly signed callback with whitespace or escaped unicode was rejected with 401. This is the incident log's "the incoming signature check has its own problems". **Root cause:** the HMAC was computed over `JSON.stringify(parsed body)`, not the bytes received. **Fix:** the callback route keeps the raw body and verifies that. **Verified:** a pretty-printed, escaped body signed over its raw bytes returns 200; a tampered body returns 401; malformed JSON returns 400.

F90, F91, F92 — Smaller correctness gaps

F90: credits accepted a future effective date, although the contract rejects it, so ledger legs were dated in a period that hasn't happened yet. Now dates after today's Bahrain date return 400. **F91:** split payouts never updated the lifetime payout estimate; fixed. **F92:** the admin console's default run-batch date was two days ago between 00:00 and 03:00 Bahrain time; it now defaults to the last completed Bahrain day.

F93 — Every checkout scanned the whole ledger

Symptom: under load Postgres burned CPU while the api waited on `COMMIT`, and checkouts got slower as the ledger grew. **Found:** sampling `pg_stat_activity` during the load test, then `EXPLAIN ANALYZE`. **Root cause:** the deferred balance trigger from migration 0007 runs `SELECT … WHERE entry_group_id = $1` for every ledger row at commit, and nothing indexed `entry_group_id`. That meant four full scans per capture (39 ms at 68k rows, linear in table size). The refund and discount paths also read legs by the unindexed `transaction_id`. **Fix:** migration 0080 adds both indexes. **Verified:** the plan switches to an index scan (39 ms → 0.26 ms) and the load test holds about 55 checkouts/s. **Existing data:** none changed. On a large production ledger, build the indexes `CONCURRENTLY` first; the migration is then a no-op.

> **F94 — Production login could never complete**
>
> **Symptom:** in production no one could receive a one-time code. The OTP "notifier" only wrote a log line, and production left the code out of it on purpose. The only configurations where login worked returned the code in the login response, which is the case file's "login works without the OTP". **Fix** (commit `4e85bda`): codes are emailed over SMTP. Production without SMTP refuses login (503 `otp_delivery_failed`) and revokes the pending session, and production never returns the code, even with `OTP_DEV_EXPOSE=true`. **Verified:** on a throwaway database, production login without SMTP returns 503 with no code and leaves no live session; development still returns the code. Delivery through a real relay needs the provider's credentials on the server.

Platform volume counter (item 73, partly fixed)

In Go, `current + int32(gross)` wrapped negative past 2,147,483,647 fils. That is only about 147,483 BD after the 2,000,000,000-fils volume tier switches on, so the 15 bps bonus would silently switch back off. The counter now stops at its maximum instead of wrapping; widening the INTEGER column still needs a DBA-approved migration.

Items 1–74 (first rounds), summary

Group	Items	Status
Auth, OTP, impersonation, admin roles, KYC, API keys	4–6, 12–15, 18–20, 54, 60–61	Fixed
Callbacks, webhooks, payout notices (FP-201, FP-240, 08-21)	7–9, 22, 55–57, 64	Fixed
Config and Docker (prod in dev mode, destructive seed on `up`)	10–11, 67	Fixed
Payment-link lock starving the pool (05-14 follow-up)	21	Fixed
Refunds / FP-300 end to end	30–38	Fixed (v1 501 stub kept by instruction)
Settlement: scheme residue, splits, paid-batch MDR, fee rates, daemon timing, test mode	42–46, 48, 62–63, 65–66	Fixed
Balances, reporting, reconciliation	39–41, 51–53, 69	Fixed
March sweep, CSV import, seed	23–29, 49–50, 59, 68	Fixed
Retained by explicit instruction or approval needed	1–3, 16–17, 47, 71–72, 74; 73 partly fixed	Left as is (see Decisions)
Open / no change needed	70 (commit-order outbox); 58 (jitter is correct)	Open / n/a

Root causes

Underlying cause	Findings it explains
Duplicate implementations that drifted apart. The same figure is derived twice and nothing reconciles the two.	Balance button vs CLI (52), analytics rebuild vs worker (F79), capture fee vs settlement fee (62–63, F88), disbursement record via webhooks (F77), fee display vs rate in effect (46)
Check-then-act without a lock or a unique constraint.	Concurrent refunds (31), duplicate payout notices (22, 64), OTP guess limit (5–6), duplicate imports (27–28),

	hold expiry SELECT/DELETE (F80)
Wrong unit or wrong clock. Two-decimal assumptions in a three-decimal currency; UTC where the business day is Asia/Bahrain.	CSV 10× (49), discount scaling (50), March window (23), daemon at 03:15 (65), reporting dates (39), admin default date (F92)
Trusting input. Client clock, re-serialized bodies, unchecked keys and enums.	Expired link paid via client clock (FP-187), unsigned or replayed callbacks (7–8), inbound signature (F89), F83–F87
Missing authorization at the route.	F75, F76, impersonation scope (12–15), bulk KYC (18–19), void role (54)
Process. Agent instruction files (`CLAUDE.md` , `AGENTS.md` , `.cursorrules`) state false facts: amounts in "cents", UTC settlement windows, "docs are truth". Locked tests pin the defects.	Explains why earlier AI-assisted changes kept reintroducing bugs (see section 6)

Decisions and open risks

Not changed	Why	Proposed safeguard
`Math.random()` for OTPs, invite codes, link slugs (1–3)	Explicit repository instruction	Switch to `crypto.randomInt` ; it's a one-line change once policy allows
Credit approval compares display names (16–17)	Explicit instruction	Compare `admin_user_id` (the handover's own rule)
INTEGER platform counter, REAL lifetime estimate (73–74)	Needs a DBA-approved migration	Migrate both to BIGINT fils; the clamp (73) prevents the tier flip meanwhile
Outbox read by id, not commit order (70)	Design change	Use the `xid8` / `pg_snapshot` query mentioned in the handover
Locked tests asserting old behaviour (7 API, 2 Go, 1 script)	Tests must not be edited	Replace them with tests that assert the ADRs
Live demo over plain HTTP on an IP address	No domain name yet, so no TLS certificate	Put the edge behind a domain with TLS (e.g. Caddy with Let's Encrypt), then run with secure cookies
Historical data (F77 duplicates, F78 stranded captures, F88 double credits)	Fixes are forward-only; nothing was run on real data	Read-only detection queries first, then finance-approved corrections

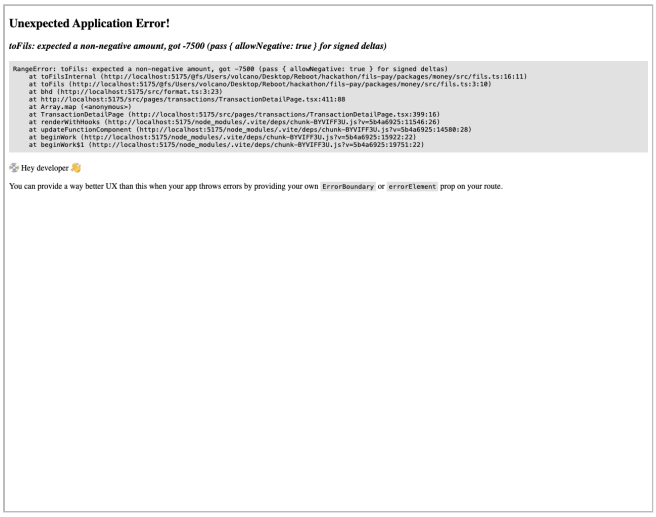
Plausible, not confirmed:

- No private-address check on webhook URLs (SSRF): "send test event" reflects the response body.
- One malformed outbox event stops the reporting worker; it needs a dead-letter policy.
- Recalculate MDR re-prices at the flat `mdr_bps` and ignores fee schedules; the locked test name suggests this is intended.
- The capture route has no merchant role gate.
- Password login has no rate limit.

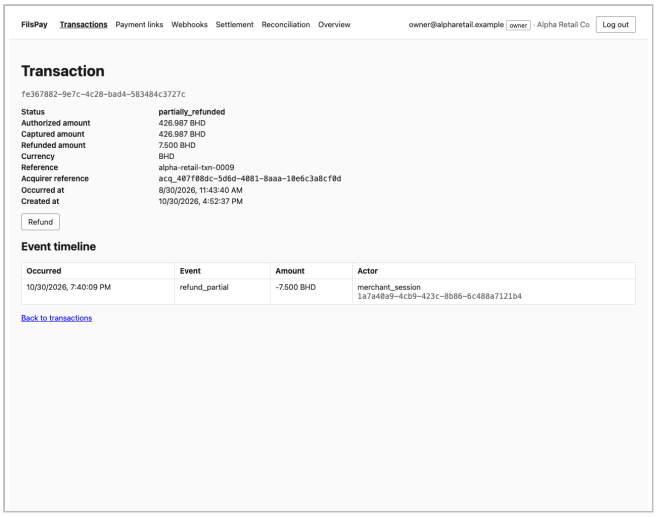
5. UI and UX changes

Refunded transactions and reconciliation no longer crash the dashboard (F81)

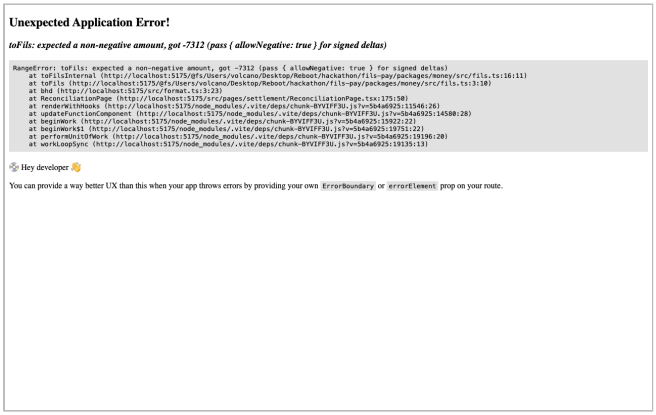
Refund timeline rows are negative by design, and a reconciliation gap or pending figure can be negative. The dashboard's money formatter refused negative amounts, so after FP-300 shipped, every refunded transaction and the reconciliation page showed an error screen instead of data. The screenshots show the same session, the same data and the same partial refund (7.500 BD back on a 426.987 BD capture), with only the formatter changed.



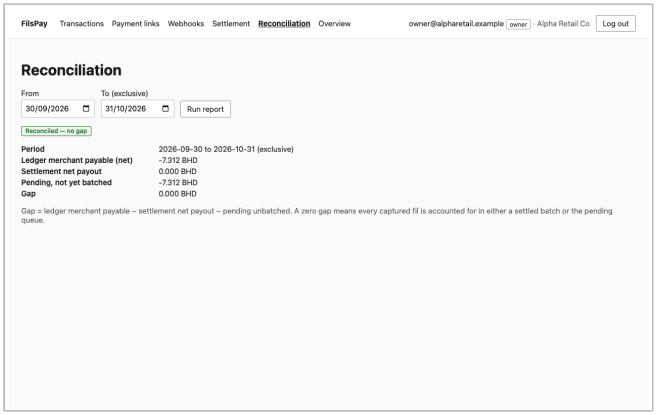
Before: transaction detail crashes on the –7.500 BD refund row



After: status partially_refunded, refund row –7.500 BD



Before: reconciliation crashes on a –7.312 BD pending figure



After: the refund reconciles, gap 0.000 BD

Other user-facing fixes without a visual difference

Change	Why it mattered	Evidence
Session cookie lasts the session's 12 h (F82)	Merchants were signed out 10 minutes after every login	The verify response now sets `Max-Age=43200`
KYC documents with Arabic filenames download (F85)	The download failed with a 500	RFC 6266 `filename*` header; reproduced before and after
Admin run-batch defaults to the last completed Bahrain day (F92)	Between 00:00 and 03:00 it defaulted to two days ago	Date arithmetic in UTC+3

Checkout under load: overloaded or restarting api answers 503 + Retry-After	Buyers saw slow 500s or 502s	Section 3

[Ali Aldoseri: visual design changes to checkout, dashboard and admin console, with before and after screenshots]

6. AI usage

Setup. Claude Code (VS Code extension) with read and write access to the repository, a disposable Postgres 16 container (`filspay-test`) with one fresh database per verification run, and the Go and Node toolchains. Every fix was verified on an untouched export of `HEAD` plus only our patch, so concurrent edits in the shared working tree could not affect results. The team used three assistants (Claude Code, Codex and Gemini CLI) and gave them one shared rule set. All of it is committed:

File	What it does
`AGENTS.md`	The shared rules, read directly by Codex. Correct facts (fils, the Bahrain business day, discount order, ledger-based balances), what must not change without sign-off, the list of locked tests that fail on purpose (so no agent "fixes" one by reverting a bug fix), and the working rules: leads are not proof, reproduce before fixing, never seed a real database, never claim an unrun test passed.
`CLAUDE.md`, `GEMINI.md`	Import `AGENTS.md` for Claude Code and Gemini CLI, so all three tools follow the same rules.
`.claude/settings.json`	Claude Code permission rules. **Deny:** editing `packages/money`, its Go mirror or any test file, force-pushes, `git reset --hard`. **Ask first:** seeding, committing, pushing, `rm`. **Allow:** tests, typecheck, read-only git.
`.claude/commands/`	Custom commands. `/verify` runs the full verification and classifies every failure. `/triage-finding` traces, reproduces (including against the original commit) and classifies a suspected defect. `/load-test` runs the checkout load test plus the integrity check.
`scripts/dev/verify.sh`	Helper script. Creates a brand-new throwaway database, runs migrations, typecheck, every TypeScript suite, the scripts suite and `go vet`/`go test`, and prints one line per step.
`.cursorrules`	Kept only as a pointer to `AGENTS.md`. Its old contents, like the old `CLAUDE.md`, stated the planted false rules.

We deliberately did *not* follow the repository's own agent files where they contradict the ADRs: "amounts are cents", "settlement windows are UTC", "docs are the source of truth".

How we prompted the review, and why

We did not ask the AI to "find bugs". We gave it one long, structured brief, written to keep it on the rails the repository itself kept knocking it off. The brief, condensed:

```
Review the prior bug-finding work, find additional real defects, fix the ones you are
authorised to fix,
and verify the result. Do not stop because you found a number of bugs or because it takes
time.

First, read completely: the agent instructions, ARCHITECTURE.md, every ADR, HANDOVER.md,
CHANGELOG.md,
```

```
the bug inventory and the earlier reports. Treat prior reports as leads, not proof.

Hard constraints: don't touch packages/money or its Go mirror; don't edit tests (they are
regression-locked;
never change expectations to make a test pass); use parseBhdToFils for amounts; keep the v1
501 stubs, the
deferred refund webhook, the documented Math.random() use and the display-name comparison;
don't widen money
columns without an approved migration; keep settlement windows consistent with the
architecture and ADRs;
no deploys, pushes or messages to external systems; never claim a test passed unless you ran
it.

For each suspected defect: trace input to side effect; check auth, validation, transactions,
concurrency,
idempotency, time zones, money units, constraints; check the docs for intended behaviour;
classify it as
confirmed, plausible, intentional or false positive. Only confirmed defects count. Don't use
a finding count
as a goal.

Fix every confirmed defect you safely can, with small focused changes. If a fix is prohibited
or needs
approval, leave it, say why, and carry on.

Verify: git diff --check, typecheck, npm test, go test, migrations on disposable databases
only. Tell
regressions apart from stale assertions, flaky tests and shared-database contamination.

Deliver: fixes in the tree, an updated inventory with separate totals (fixed, open with
reason, intentional,
test/environment problems), and a candid report. Don't claim the code is bug-free.
```

Why it is written that way:

Part of the brief	Failure it guards against
Read the ADRs, architecture and handover *before* editing	The repository's own agent files (`CLAUDE.md`, `AGENTS.md`, `.cursorrules`) contain confident, false rules: "amounts are cents", "settlement windows are UTC", "the Go service is always right". A model that obeys the first instruction file it reads repeats those bugs. Reading the decision records first lets it notice the contradiction, and the ADRs win.
Prior reports are leads, not proof	Earlier rounds produced confident claims that did not hold up, for example "the Go tests pass", which was only true because the database tests skip silently without `DATABASE_URL`. Every claim has to be re-checked against code and a real database.
Explicit hard constraints	Without them a model "fixes" intentional behaviour (the v1 501 stubs, the deferred refund webhook) or edits frozen, audited code to make a symptom disappear.
Never edit tests; never change expectations to pass	The commonest way an AI turns red into green is to change the test, or change correct code to match a test that pins a bug. Several locked tests here assert the defects themselves, so a failing test has to be explained, not silenced.
Classify every lead; only confirmed defects count; no finding-count target	Prevents padded reports and plausible-sounding but invented bugs. Each finding has to survive a trace through the code and, wherever possible, a reproduction.
Decide routine matters alone, but stop at policy lines and explain	A long autonomous run should not stall on questions it can answer from the docs. It also must not cross a line (a schema widening, a frozen file) that needs a human's sign-off.
Verification on disposable databases; distinguish	"Tests fail" carries no information unless you know whether it is a regression, a stale assertion or one suite polluting another's data. It also keeps real data out of reach.

failure types	
Don't claim a test passed unless it ran; be candid about coverage	Directly targets hallucinated verification, the failure that matters most in a payments codebase.
No deploys, pushes or external messages	Keeps every outward-facing action with the team.

The same brief also explains the follow-up checks we asked for: "is everything finished?" (which turned five "plausible" leads into reproduced, fixed defects) and "are these bugs from the original repo or from us?" (answered by running the reproductions against the original commit).

Excerpt 1 — F80, hold expiry deletes every hold

Prompt and context: a broad review prompt (review prior work, find new defects, fix, verify, don't edit tests) with the full repository, handover and ADRs. **AI answer (excerpt):**

```
Critical finding: scripts/expire_holds.ts runs `DELETE FROM holds` with no WHERE clause on
any night
with no expired holds, because buildWhere drops empty arrays. That wipes every hold, active
ones
included. It also deletes rows the handover says must only be flipped to `expired`.
```

How we verified: read `sqlFilter.buildWhere` (empty array → `continue`), then ran the patched script on a database with one live hold and zero expired holds: `matched 0 … deleted 0 row(s)`, and the live hold remained. With one expired hold, only that one was deleted. **What we changed:** empty list → `FALSE`; skip the delete when nothing matched; one transaction with `FOR UPDATE`. The AI kept the locked test's `DELETE … ANY($1)` text unchanged.

Excerpt 2 — F88, discount double credit

Context: in its first report the AI listed this as only "plausible but unconfirmed". When we asked "is everything finished?", it went back and reproduced it. **AI output (excerpt):**

```
statuses 201 200 200 after two captures: { n: 1, s: 2500 }
after replaying capture 1: { n: 2, s: 5000 }
Confirmed, and it's a real money defect. Replaying the first capture's idempotency key
credits the
merchant the second capture's 2.500 BD discount a second time.
```

Verified: after the fix, the same script printed `after replaying capture 1: { n: 1, s: 2500 }`, and the discount/refund/transaction suites passed apart from the one pre-existing stale assertion. **Lesson:** labelling something "plausible" is not the end of the job; we required a reproduction before accepting or rejecting each lead.

Excerpt 3 — the AI was wrong about its own verification

What happened: to test "our patch only", the AI generated a patch from a list of changed files and ran the suites on a clean copy. In zsh, `git diff -- $MINE` does not split the variable into separate paths, so the first patch was empty. A later attempt filtered a one-line file list with `grep -v`, which dropped every file. For a while, a "patched" copy contained only 2 of 16 fixes, while the AI described it as the full patch. **How it was noticed:** before building the submission, the AI compared every changed file in the clean

copy against the working tree. The check printed `differs from live` for 16 files and the patch held only 8 diffs. **What we changed:** rebuilt the clean tree file by file, confirmed it was byte-identical to the patch result (`WT3_EQUALS_MINE`), and re-ran every suite and reproduction on it. A related catch: a previous AI-written inventory said `go test ./...` passed. The database-backed Go tests silently skip when `DATABASE_URL` is unset, and with a real database two tests fail.

Appendix

Test results — clean `HEAD` + our patch, fresh migrated database (0001–0079)

Check	Result	Notes
`git diff --check`, `npm run typecheck`, `go vet`	clean	
money / events / web	40/40, 4/4, 8/8	
API	355/362	7 locked tests assert pre-fix behaviour (4.75 BD = 475 fils, discount off the fee, fee display, exact jitter)
Reporting	7/11 combined, 11/11 alone	Shared-database contamination from API tests
Scripts	4/5	Locked test expects the UTC month; ADR-0007 requires the Bahrain month
Go	all packages ok except `ledger` (2)	`refresh_test.go` asserts the pre-ADR-0008 balance source (FP-203)

The same set fails on the unmodified baseline: our changes add no new failures. Logs are in `submission/evidence/`.